

Steps:

1. Identify problem statement(Statement of purpose)
2. Requirement gathering and specification
 - a. Requirement gathering and analysis
 - i. Requirement gathering
 - ii. Requirement analysis
 - b. Requirement Specification

❖ Software Requirement Specification(SRS)

The Software Requirements Specification (SRS) is a structured document that systematically organizes all user requirements after removing ambiguities, inconsistencies, and incompleteness. (blackbox specification of s/w)

After gathering and analyzing requirements (removing **ambiguities, inconsistencies, incompleteness**), the analyst **organizes** them into an **SRS document**.

SRS contains **all user requirements** in a **structured but informal** form.

It is **one of the most important and most difficult documents** to write.

It must satisfy the needs of **many different audiences**.

It describes what the software should do in a clear, precise, and implementation-independent manner.

It acts as a contract between the customer and the development team.

Whatever we cleaned in requirement analysis for the Library – we now write properly inside the SRS

• Users of SRS Document

The SRS is used by **many categories of people**, each for different purposes:

For our Library System, the SRS is not just for the developer. It is the **single reference** for everyone in the project.”

1. **Users / Customers / Marketing personnel** – to check if the system will meet their needs.

Users / Customer (Librarian / Management)

- They read the SRS to see:“Does this system support search by title/author? Can we handle overdue books? Can we get the reports we need?”
2. **Software developers** – to know *what exactly to build*.
- They use the Library SRS to implement: IssueBook, ReturnBook, SearchBook, CalculateFine functions exactly as described.

3. **Test engineers** – to design **test cases** from the specified functionality, inputs, and outputs.

They design tests like:

- “When member enters book title and it exists, system should show its availability and location.”
- “If user has already issued max 3 books, issuing a 4th must be rejected.”

4. **User documentation writers** – to write user manuals.

They use the SRS to write:

- “How to search a book”, “How to renew a book”, “How to pay fines” in the user manual

5. **Project managers** – to estimate **cost, time, resources, schedule..**

From the SRS, they estimate:

- Number of screens, reports, modules → effort, cost, timeframe.

6. **Maintenance engineers** – to understand existing functionality and plan changes.

When college wants to add **e-book integration** later, they first read the SRS to understand what functionality is already there.

❖ **Why Spend Time and Resources to Develop an SRS?**

A good SRS finds following usage:

For our Library example, a strong SRS saves time, reduces fights, and makes future changes easier.

1. **Forms an agreement** between customer and developers.

Example: Library SRS clearly says:

- “Students can issue max 3 books”,
- “Fine is ₹2/day after due date”,
- “Search supports title, author, and subject.”

Later, if librarian says “*I also wanted video CDs*” but not written in SRS, it’s clear this is a **new requirement**, not a mistake.

2. **Reduces future rework** – because requirements are thought through before design and coding.

Example: If we clarify early that **reference books cannot be issued**, developers won't waste time implementing issue-logic for reference books and then removing it later.

3. **Provides a basis for estimating cost and schedule.**

Example: From Library SRS:

- No. of screens (login, search, issue, return, reports, admin, etc.)
- No. of reports (daily issue, overdue list, fines, etc.)
- Project manager uses this to estimate **time, team size, and budget**.

4. **Provides a baseline for validation and verification.**

Example: Test engineers check the final Library System against SRS:

- *"Does it allow renewal only if no one has reserved the book, as written?"*
- *"Does it show all overdue books correctly?"*

5. **Facilitates future extensions.**

Example: After 2–3 years, college wants: **Mobile app access**, or **Integration with digital library (e-books)**.

SRS helps understand current system clearly for planning these enhancements.

❖ **Characteristics of a Good SRS Document**

The SRS document should describe the system (to be developed) as a black box, and should specify only the externally visible behavior of the system.

For this reason, the SRS document is also called the black-box specification of the software being developed.

Good SRS for our Library System must be: clear, testable, not talking about internal design, and easy to update.

Good qualities of SRS documents are following.

1. **Concise** – short but **unambiguous, consistent, and complete**.

- Good: "Student can issue maximum **3 books** at a time."
- Bad: one long paragraph mixing rules, history, and stories.

2. **Implementation-independent** – focuses on *what* the system should do, not *how* to do it (black-box view).

- Good SRS statement: “System shall store details of each member and allow searching by membership ID.”
- Bad : “Store member data in a MySQL table with index on ID and use binary search”
- That “how” part belongs to design, not SRS.

3. **Traceable** – each requirement can be linked to design, code, and test cases, and vice versa.

Give requirement IDs:

- R1: Search book
- R1.1: Select search option
- R1.2: Enter keywords and display results

Later, in design and code, we can say:

- Class SearchService implements R1, test case TC-101 tests R1.2.

4. **Modifiable** – easy to change because it is **well-structured** and not scattered.

- If library changes rule from **3 books** to **4 books**, this should be written at **one place** in SRS, not scattered in 10 different pages.
- A well-structured SRS (with sections like “Issue Policy”, “Fine Policy”) makes change easy.

5. **Identifies responses to undesired events** – handles error/exception conditions.

- Example Library SRS should specify:
 - What happens if a student enters invalid member ID?
 - What if the book is already issued?
 - What if the database connection fails?

These are all described under exception scenarios.

6. **Verifiable** – requirements are written in a way that we can **check** (test) if they are satisfied.

- Not good: “System should be **user friendly**.” (cannot test)
- Good: “System should display search results within **2 seconds** for up to **100,000 books**.”
- For Library, “When user enters a valid title, system shall display list of matching books with title, author, year, location.” → this is verifiable.

❖ Attributes of Bad SRS Documents

Besides **ambiguity, incompleteness, and contradiction**, common problems are:

1. **Over-specification** – SRS goes into “how to implement”.

Bad SRS: “Library must store books in a table tblBooks with B-tree index on title and use merge sort for listing.” This is **design/database detail**, not requirement.

Good SRS: “System shall allow searching books by title, author, and category.”

2. **Forward references** – referring to things explained much later, reducing readability.

Bad:

- In early section: “Book reservation feature as explained in Section 9.4.7.2 will handle all edge cases.”
- But Section 9.4.7.2 is 30 pages later.
- This confuses readers.

Good: give a **short description here** and refer to details later

3. **Wishful thinking** – specifying features that are **very hard or unrealistic** to implement.

Bad SRS requirement:

- “System shall automatically understand students’ reading interests using AI and recommend books without any configuration or training.”
- Sounds nice but may be **difficult or unrealistic** for current project scope and budget.

4. **Noise** – extra irrelevant information, not useful for software, which clutters the SRS

- Bad: “The library building is 40 years old and has 3 floors. Librarian reaches office at 9 AM on a scooter...”
- Interesting, but **not relevant** for the software behavior.
- It just increases page count and distracts developers.

❖ GOOD SRS vs BAD SRS (Key Point Comparison + Library Examples)

Key Point	Good SRS	Bad SRS	Library System Example
1. Clarity & Unambiguity	Requirements are written in clear, measurable terms.	Uses vague or subjective words.	Good: "Student can issue maximum 3 books. " Bad: "Student can issue few books."
2. Completeness	Covers all required features and data elements.	Missing important details or steps.	Good: SRS includes fields: Title, Author, ISBN, Location Bad: SRS says "Search books" but does not define what book information is stored.
3. Consistency	No contradictions between requirements.	Conflicting or opposite requirements.	Good: "Reference books cannot be issued." Bad: One section says reference books can be issued for 1 day; another says cannot be issued.
4. Implementation-independent	Describes what to do, not how to do it.	Contains design or technical decisions prematurely.	Good: "System shall store member information." Bad: "Store member data in a MySQL table with B-tree indexing."
5. Conciseness	Short, focused, no extra stories.	Verbose, unnecessary details (noise).	Good: "System shall generate overdue book report." Bad: "The library is 40 years old, and the librarian arrives at 9 AM..."
6. Traceable	Requirements numbered; easy to trace to design, code, tests.	No numbering or mixing; hard to link to implementation.	Good: R1 Search Book → Design component D1 → Test case TC1. Bad: Requirements scattered randomly with no IDs.
7. Modifiable	Well-structured; changes affect only one place.	Requirements repeated or scattered; hard to update.	Good: "Issue policy" written in ONE section. Bad: "Max books = 3" written in five different

Key Point	Good SRS	Bad SRS	Library System Example
			places; changing it requires editing all.
8. Verifiable	Testable statements; can create test cases.	Non-testable, subjective statements.	Good: "Search results must appear in < 2 seconds." Bad: "System should be fast and user friendly."
9. Handles exceptional/undesired events	Specifies system response in error/edge cases.	Ignores exceptions.	Good: "If a book is already issued, system shall display 'Not Available'." Bad: SRS only describes normal issuing, not what happens when book is missing.
10. Avoids Wishful Thinking	Only realistic requirements.	Includes unrealistic/impossible features.	Good: "System shall send email reminders for overdue books." Bad: "System shall automatically understand students' reading habits using AI without any extra data."